

Assignment of Homework 1 (deadline: April 11)

The purpose of the first homework is to get acquainted with Spark and with its use to implement MapReduce algorithms. In preparation for the homework, you must perform and test the set up of your machine, following the instructions given in this site. The homework concerns a variant of the classical Lloyd's algorithm for k-means clustering, which enforces a fairness constraint on the solution based on extra demographic information attached to the input points.

K-MEANS CLUSTERING. Given a set of points $U \subset \mathbb{R}^D$ and an integer K , k-means clustering aims at determining a set $C \subset \mathbb{R}^D$ of K centroids which minimize the following objective function:

$$\Delta(U, C) = (1/|U|) \sum_{u \in U} (\text{dist}(u, C))^2$$

where $(\text{dist}(u, C))^2$ denotes the **squared Euclidean distance** between u and the centroid of C closest to u . Since the problem is NP-hard, current efficient solutions seek approximate solutions. Lloyd's algorithm is widely used to this purpose. It works as follows. Let M be an integer parameter:

1. Compute an initial set $C = \{c_1, c_2, \dots, c_K\}$ of centroids
2. **Repeat M times**
 - 2.1. Partition U into K clusters $U_1, U_2, \dots, U_K$, where U_i consists of the points of U whose closest centroid is c_i (*assume that ties are broken in favor of the smallest index*).
 - 2.2. For $1 \leq i \leq K$, compute a new centroid c_i as the average of the points of U_i

Note that, rather than using a user-defined parameter M to specify the number of iterations, suitable convergence conditions are often employed to decide when to stop the iterations.

SPARK IMPLEMENTATION of Lloyd's ALGORITHM. In the RDD-based API of the mllib package, Spark provides an implementation of LLoyd's algorithm for k-means clustering. In particular, the algorithm is implemented by method train of class KMeans which receives in input the points stored as an RDD of Vector in Java, and of NumPy arrays in Python, the number k of clusters, and the number of iterations. The method computes an initial set of centroids using, as a default, algorithm kmeans|| (a parallel variant of kmeans++), and then executes the specified number of iterations. As output the method returns the final set of centroids, represented as an instance of class KMeansModel. For this latter class, method clusterCenters will return the centroids as an array of Vector in Java and list of NumPy arrays in Python. Refer to the official Spark documentation on [clustering.\(RDD-based API\)](#) for more details.

FAIR K-MEANS CLUSTERING. A fair variant of Lloyd's algorithm was proposed in the paper [Socially Fair k-Means Clustering.\(ACM EAccT'21\)](#), by M. Ghadiri, S. Samadi, and S. Vempala. Suppose that the input set of points U is split into two **demographic groups** A, B (e.g., corresponding to female and male individuals). That is $U = A \cup B$. The new algorithm addresses a socially fair version of k-means where the set C of centroids aims at minimizing the following objective function:

$$\Phi(A, B, C) = \max\{(1/|A|) \sum_{a \in A} (\text{dist}(a, C))^2, (1/|B|) \sum_{b \in B} (\text{dist}(b, C))^2\}$$

The new algorithm is a simple variation of Lloyd's algorithm. The only difference concerns Step 2.2, where the computation of the new centroids $c_1, c_2, \dots, c_K$ from the current partition $U_1, U_2, \dots, U_K$ is performed through a gradient descent protocol. You will implement this fair variant of Lloyd's algorithm in Homework 2, while in this homework you will be only concerned with the new objective function.

REPRESENTATION OF POINTS

- **(JAVA Users)** Points must be represented as instances of the class **Vector** of package **mllib.linalg** and can be manipulated through static methods offered by the class **Vectors** in the same package. For example, method **Vectors.dense(x)** transforms an array x of double into an instance of class Vector, while method **Vectors.sqdist(x,y)** computes the squared Euclidean distance between two instances, x and y, of class Vector. Details on these classes can be found in the [Spark Java API](#).

Warning. Make sure to use the classes from the org.apache.spark.mllib package. There are classes with the same name in org.apache.spark.ml package which are functionally equivalent, but incompatible with those of the org.apache.spark.mllib package.

- **(PYTHON Users)** Points must be represented as tuples of float (i.e., point = (x1, x2, ...)). Although Spark provides the class Vector also for Python (see pyspark.mllib package), its performance is very poor and its more convenient to use tuples, especially for points from low-dimensional spaces.

INPUT FORMAT. To implement the algorithms assume that the input set U is given in input as a file, where each row contains one point stored with the coordinates separated by comma (',') and, at then end, a character, A or B, representing the point's demographic group. For instance, a point $p = (1.5, 6.0) \in \mathbb{R}^2$ of group A, will occur in the file as line: **1.5,6.0,A**.

ASSIGNMENT for HW1. For a given input set of points U , split in two demographic groups A, B , you must compare the values of the two objective functions $\Delta(U, C)$ and $\Phi(A, B, C)$, where the set C of centroids is computed with the standard Lloyd's algorithm, ignoring the demographic group attributes. Also, you will print some statistics concerning the distribution of the two groups among the clusters. You must assume that the input set U is potentially very large, therefore it must be stored into an RDD, whose elements are pairs (p, g_p) , where $p \in \mathbb{R}^D$ is a point and $g_p \in \{A, B\}$ is a character representing its demographic group.

Specifically, for the homework you must do the following tasks.

1) Write a method/function MRComputeStandardObjective that takes in input an RDD of (point,group) pairs (representing a set $U = A \cup B$), and a set C of centroids, and returns the value of the objective function $\Delta(U, C)$ described above, thus ignoring the demopgraphic groups.

2) Write a method/function MRComputeFairObjective that takes in input an RDD of (point,group) pairs (representing points of a set $U = A \cup B$), and a set C of centroids, and returns the value of the objective function $\Phi(A, B, C)$ described above.

3) Write a method/function MRPrintStatistics that takes in input an RDD of (point,group) pairs (representing points of a set $U = A \cup B$), and a set C of centroids, and computes and prints the triplets (c_i, NA_i, NB_i) , for $1 \leq i \leq K = |C|$, where c_i is the i -th centroid in C , and NA_i, NB_i are the numbers of points of A and B , respectively, in the cluster U_i centered in c_i .

4) Write a program GxxHW1.java (for Java users) or GxxHW1.py (for Python users), where xx is your 2-digit group number (e.g., 04 or 25), which receives in input, as command-line arguments, a path to the file storing the input points, and 3 integers L, K, M , and does the following:

- Prints the command-line arguments and stores L, K, M into suitable variables.
- Reads the input points into an RDD of (point,group) pairs -called **inputPoints**-, subdivided into L partitions.
- Prints the number N of points, the number NA of points of group A, and the number NB of points of group B (hence, $N = NA + NB$).
- Computes a set C of K centroids by using the Spark implementation of the standard Lloyd's algorithm for the input points, disregarding the points' demographic groups, and using M as number of iterations.
- Prints the values of the two objective functions $\Delta(U, C)$ and $\Phi(A, B, C)$, computed by running MRComputeStandardObjective and MRComputeFairObjective, respectively.
- Runs MRPrintStatistics.

OUTPUT FORMAT: In the same section as this page you will find some datasets and outputs corresponding to specific input configurations using these datasets. Your program must **STRICTLY ADHERE TO THE OUTPUT FORMAT** used in these examples. **Any deviation from this format will be penalized.**

5) Test your program using the examples provided in the same section as this page.

IMPORTANT WARNING. The input set U as well as any intermediate data of size similar to $|U|$ can be store exclusively in distributed form, using RDDs. Otherwise, the evaluation will be penalized. One the other hand, you can safely assume that K is small enough that K points, or, in general, collections of size K , can be stored into local structures (e.g., arrays, lists, etc.)

SUBMISSION INSTRUCTIONS. Each group must submit a **zip** file containing its program (**GxxHW1.java** or **GxxHW1.py**) *Please use a zip even for single and small files*. Only one student per group must submit the file in Moodle Exam using the link provided in the Homework 1 section. Make sure that your code is free from compiling/run-time errors and that you use the file/variable names in the homework description, otherwise your score will be penalized.

If you have questions about the assignment, contact the teaching assistants (TAs) by email to bdc-course@dei.unipd.it. The subject of the email must be "HW1 - Group xx", where xx is your group ID. If needed, a zoom meeting between the TAs and the group will be organized.